

Transport State — a MISU refactor (design cut)

Companion to [STATE-AUDIT-BACKLOG.md](#). That doc named the loose threads; this one proposes the model that removes them. The thesis: the whole transport/authority surface collapses to **two stored values plus a pure function**, and almost every boolean the audit flagged is *derived*, not stored — so the illegal states can't be written down.

This is **entirely frontend**. No backend change (the rig stop verbs already exist and are committed). The core is a small **pure module** ([Triggerfish.Transport](#)) consumed by the existing Halogen root and the four child components — not a new component. The root's control loop *shrinks*: [reconcileRig](#) disappears.

1. The one authoritative value

The entire user-facing transport is:

```
-- lives in the shell (Triggerfish.Main.RState), and NOWHERE else
mode  :: Mode          -- Solo | Atlantis  (already an ADT — the one clean
bit)
armed :: Set Which     -- which machines the user has armed
```

[Set Which](#) replaces the [{odo,bal,sel,vet}](#) record *and* the asymmetric [{odo,bal,vet}](#) rigOn record. A set has no shape to get wrong: Selene being rig-less is not a missing field, it's a static fact (below).

Static, not state:

```
-- Does this instrument have a BEAM voice that can take authority?
hasRigVoice :: Which -> Boolean
hasRigVoice = case _ of
  Odo -> true
  Bal -> true
  Vet -> true
  Sel -> false    -- frontend-only POLYTRIG drums; never rig-authoritative
  Tid -> false    -- the aggregate tab, not a machine
```

2. Everything else is derived

```
-- WHERE an instrument's PERFORMANCE output is sounding right now.
data Sounding = Silent | Local | Rig
derive instance Eq Sounding

soundingOf :: Mode -> Set Which -> Which -> Sounding
```

```
soundingOf mode armed w
| not (Set.member w armed) = Silent
| otherwise = case mode of
    Solo      -> Local
    Atlantis -> if hasRigVoice w then Rig else Local

anyArmed :: Set Which -> Boolean
anyArmed = not <<< Set.isEmpty
```

That function is the whole authority model. Read against the audit's illegal-states list:

Audit's illegal / incoherent state	Why it's now unrepresentable
<code>master = false</code> anywhere	<code>master</code> is deleted . Not folded — gone.
<code>audible = true</code> on a rig instrument in Atlantis	<code>audible</code> isn't stored; it's <code>soundingOf ... == Local</code> , which is <code>Rig</code> here by construction.
<code>rigOn.x = true</code> while <code>mode = Solo</code>	there is no <code>rigOn</code> ; the rig target set is <code>soundingOf ... == Rig</code> , empty in Solo.
<code>rigOn</code> shape $\neq$ <code>armed</code> shape	one <code>Set Which</code> ; Selene filtered by <code>hasRigVoice</code> , not by a hand-kept record.
<code>playing</code> drifting from <code>armed</code>	<code>playing</code> is deleted ; the button reads <code>anyArmed armed</code> .
Selene special-cased in <code>broadcastAudible</code>	Selene's Local-in-both-modes falls out of <code>hasRigVoice Sel = false</code> . No special case.

Six flagged states, gone by construction, plus three deleted fields (`master`, `audible`, `playing`) and one deleted record (`rigOn`).

3. The query surface collapses 6 → 2

Today the shell drives instruments with six transport queries: `SetMaster`, `SetAudible`, `SetArm`, `SyncToRig`, `StopRig`, `AskArmed`. All six are facets of one question — *where should you be sounding?* — so they become one push and one read:

```
data Query a
= ...
| SetSounding Sounding a      -- "you are now Silent / Local / Rig"
| AskSounding (Sounding -> a) -- observe (see §5)
```

Each instrument stores **one** field, `sounding :: Sounding`, replacing `master + audible + running/armed` + Balistes' `pushed`. The handler is the same shape everywhere:

```
SetSounding s next -> do
  prev <- H.gets _.sounding
  H.modify_ _ { sounding = s }
```

```

when (prev == Rig && s /= Rig) stopRigVoice    -- leaving Rig: e.g. send
"reef-stop"
when (s == Rig)                                doHandoff        -- entering/refreshing
Rig: full push
pure (Just next)

```

- **Local-MIDI emit gate** everywhere becomes `st.sounding == Local` (was `st.master && st.running && st.audible`). The clock/animation keeps advancing regardless — lockstep co-sim unchanged; only *emission* gates.
- **Edge detection lives with the verbs.** Today the shell edge-detects (`rigOn`) but the instrument owns the rig verb — an awkward split. Putting the whole `Sounding` in the instrument reunites them: the one place that knows how to hand off / stop is the one place that detects the transition. `reconcileRig` deletes entirely.
- **Re-voicing while already Rig** (Vetula's settled-edit auto-repush) is just `SetSounding Rig` again — `s == Rig` re-runs `doHandoff`. No separate `SyncToRig`. The shell issues it on brush-settle exactly as it does now, gated on `soundingOf ... Vet == Rig` instead of `rigOn.vet`.

4. The shell control loop, after

```

-- The only writers of transport state:
ToggleMaster ->                                -- master ►/■ = arm-all / disarm-all
setArmed (if anyArmed a then Set.empty else allMachines)

ArmTab w ->                                     -- a tab dot = toggle one
setArmed (toggle w a)

SetMode m -> H.modify_ _ { mode = m } *> broadcastSounding

```

where the single helper replaces `broadcastMaster` + `broadcastAudible`

- `broadcastSyncToRig` + `broadcastStopRig` + `reconcileRig`:

```

broadcastSounding :: ... -- push soundingOf to each of Odo/Bal/Sel/Vet
setArmed newArmed = do
  H.modify_ _ { armed = newArmed }
  broadcastSounding

```

Push cadence: on every transport change (arm/mode) push all four; on Vetula brush-settle re-push Vetula only. No per-tick re-push — Odonus and Balistes still stream their own edits live over reef-input/balistes-input independent of transport.

5. The one axis that is *not* derivable — and is honestly named

`sounding` in the instrument is a **committed belief** about the rig, not pure intent. It can diverge from `soundingOf` in exactly two ways, both observation gaps, both already tolerated:

1. **Instrument self-disarms** (Vetula unloading a progression). The instrument is, for that instant, a source of truth, not a view.
2. **Rig voice dies under us** (mid-Atlantis restart). The instrument still believes **Rig**; desired hasn't changed, so no re-push.

This is the classic *desired vs observed* split — plan/apply, the Bosun pattern — and the fix for both is the same **observe** step, which we already run: **SyncTick** polls each instrument and folds the answer back into **armed**:

```
-- observe: armed := { w | AskSounding w /= Silent }
```

So **AskArmed** isn't deleted, it's **reframed** as the observation edge of a reconcile loop, and it now covers *both* gaps with one mechanism instead of the self-disarm poll and the rig-drift round-trip being separate mysteries. (Fully-MISU endpoint, noted not built: replace the poll with a child **output** — the instrument *raises* **Disarmed** and the shell folds it — so the shell is the sole writer and the divergence window closes. That needs the slots' **Void** output type opened up; bigger surgery, deferred.)

6. Where the model pushes back — the thing we learn

MISU earns its keep by refusing to represent something and making us look at why. One real friction surfaces:

Audition must sound locally even when **Sounding == **Rig**.** In Atlantis a rig instrument is muted locally — but manual chord audition (Vetula's **playChord/playPath** on **previewChan**) must *still* make local sound; that's the whole point of the preview-channel picker (#71/#73).

The lesson: **Sounding** is not "is local MIDI on," it is "**where is the PERFORMANCE output.**" Audition is an orthogonal, always-local path that **Sounding** must never gate. This is already how the code behaves (**playChord** is ungated today), but the refactor makes it a *stated boundary* rather than an accident: two output paths — performance (governed by **Sounding**) and audition (always local, ungated). If we'd tried to make **Sounding** govern *all* local MIDI, audition-in-Atlantis would have been the illegal state we couldn't build — and that's the signal that the axis was drawn in the wrong place.

No other blocker found: master-play, per-tab arm, solo/atlantis authority, Selene's hybrid, and the Vetula auto-repush all express cleanly.

7. Change inventory

New: **Triggerfish.Transport** (pure) — **Sounding**, **soundingOf**, **hasRigVoice**, **anyArmed**, **allMachines**. Add **Ord Which** (for **Set**).

Shell (Main.purs): RState transport fields → **mode :: Mode**, **armed :: Set Which** (delete **playing**, **rigOn**). Delete **reconcileRig**, **broadcastMaster/Audible/SyncToRig/StopRig**, the **ArmState** record helpers. **ToggleMaster/ArmTab/SetMode** rewritten as §4. **broadcastSounding** added. **SyncTick** observe folds **AskSounding** → **armed**. Renderers read **anyArmed/soundingOf** instead of **playing/armed.x**.

Each instrument: delete `master`, `audible`, (`pushed`); add `sounding :: Sounding`. Replace the `SetMaster/SetAudible/SetArm/ SyncToRig/StopRig` handlers with the one `SetSounding` handler (§3); `AskArmed` → `AskSounding`. Emit gates → `sounding == Local`. Selene's handler treats `Rig` defensively as `Local` (it never receives it).

SourceQuery + Vetula.SourceQuery: the six transport constructors → `SetSounding` + `AskSounding`. Delete the dead `PushVetula/PushBrush/ Hush*` actions and the retired `vetula-perf` path (audit item 6) in the same pass.

Channels (audit item 7): out of scope for this cut — it's an independent normalisation, do it separately.

8. Suggested order

1. Land `Triggerfish.Transport` pure module + a couple of unit tests on `soundingOf` (Selene-in-Atlantis = Local; disarmed = Silent; rig instrument armed in Atlantis = Rig).
2. Widen `SourceQuery` to `SetSounding/AskSounding` **alongside** the old constructors (both compile), migrate one instrument (Odonus) end to end, verify on the rig.
3. Migrate Bal / Sel / Vet, then rip out the old six constructors and the shell's `reconcileRig/rigOn/playing`/broadcasts.
4. Delete dead paths. Channels normalisation is a separate ticket.